

Búsqueda y planificación

La **búsqueda y planificación** es un conjunto de acciones para lograr encontrar la/s posible/s soluciones a un problema en concreto.

Los algoritmos de búsquedas se pueden clasificar en dos grandes grupos:

1. Búsquedas Completas o no Informadas:
 - Se asegura la consecución del objetivo si existe una solución.
 - Costosas (complejidad exponencial).
2. Búsquedas Incompletas (Heurísticas):
 - Basadas en conocimientos intuitivos o más rigurosos del problema (heurística).
 - Ahorro computacional considerable.

Búsquedas Completas o No Informadas

El término significa que ellas no tienen información adicional acerca de los estados, más allá de la que proporciona la definición del problema. Todo lo que ellas pueden hacer es generar los sucesores y distinguir entre un estado objetivo de uno que no lo es.

Búsqueda en amplitud

En Ciencias de la Computación, *Búsqueda en anchura* (en inglés BFS - *Breadth First Search*) es un algoritmo para recorrer o buscar elementos en un grafo (usado frecuentemente sobre árboles). Intuitivamente, se comienza en la raíz (eligiendo algún nodo como elemento raíz en el caso de un grafo) y se exploran todos los vecinos de este nodo. A continuación, para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el árbol.

Formalmente, *BFS* es un algoritmo de búsqueda sin información, que expande y examina todos los nodos de un árbol sistemáticamente para buscar una solución. El algoritmo no usa ninguna estrategia heurística.

Si las aristas tienen pesos negativos, aplicaremos el algoritmo de Bellman-Ford en alguna de sus dos versiones.

Procedimiento

- Dado un vértice fuente **s**, *Breadth-first search* sistemáticamente explora los vértices de **G** para “descubrir” todos los vértices alcanzables desde **s**.
- Calcula la distancia (menor número de vértices) desde **s** a todos los vértices alcanzables.
- Después produce un árbol **BF** con raíz en **s** y que contiene a todos los vértices alcanzables.
- El camino desde **s** a cada vértice en este recorrido contiene el mínimo número de vértices. Es el camino más corto medido en número de vértices.
- Su nombre se debe a que expande uniformemente la frontera entre lo descubierto y lo no descubierto. Llega a los nodos de distancia **k**, solo tras haber llegado a todos los nodos a distancia **k-1**.

Pseudocódigo

- La nomenclatura adicional utilizada es: **Q** = Estructura de datos cola.

BFS(grafo **G**, nodo_fuente **s**)

```
{
    // recorremos todos los vértices del grafo inicializándolos a
    NO_VISITADO,
    // distancia INFINITA y padre de cada nodo NULL
    for u ∈ V[G] do
    {
        estado[u] = NO_VISITADO;
        distancia[u] = INFINITO; /* distancia infinita si el nodo no es
alcanzable */
        padre[u] = NULL;
    }
    estado[s] = VISITADO;
    distancia[s] = 0;
    Encolar(Q, s);
    while !vacía(Q) do
    {
        // extraemos el nodo u de la cola Q y exploramos todos sus nodos
adyacentes
        u = extraer(Q);
        for v ∈ adyacencia[u] do
```

```

{
    if estado[v] == NO_VISITADO then
    {
        estado[v] = VISITADO;
        distancia[v] = distancia[u] + 1;
        padre[v] = u;
        Encolar(Q, v);
    }
}
}
}

```

Búsqueda en profundidad

Una **Búsqueda en profundidad** (en inglés DFS o *Depth First Search*) es un algoritmo que permite recorrer todos los nodos de un grafo o árbol (teoría de grafos) de manera ordenada, pero no uniforme. Su funcionamiento consiste en ir expandiendo todos y cada uno de los nodos que va localizando, de forma recurrente, en un camino concreto. Cuando ya no quedan más nodos que visitar en dicho camino, regresa (Backtracking), de modo que repite el mismo proceso con cada uno de los hermanos del nodo ya procesado.

- **Pseudocódigo para grafos**

DFS(grafo G)

```

PARA CADA vertice  $u \in V[G]$  HACER
    estado[u]  $\leftarrow$  NO_VISITADO
    padre[u]  $\leftarrow$  NULO

tiempo  $\leftarrow$  0

PARA CADA vertice  $u \in V[G]$  HACER
    SI estado[u] = NO_VISITADO ENTONCES
        DFS_Visitar(u)

```

DFS-Visitar(nodo u)

```

    estado[u]  $\leftarrow$  VISITADO
    tiempo  $\leftarrow$  tiempo + 1
    d[u]  $\leftarrow$  tiempo

    PARA CADA  $v \in \text{Vecinos}[u]$  HACER
        SI estado[v] = NO_VISITADO ENTONCES
            padre[v]  $\leftarrow$  u

```

```

DFS_Visitar(v)

estado[u] ← TERMINADO
tiempo ← tiempo + 1
f[u] ← tiempo

```

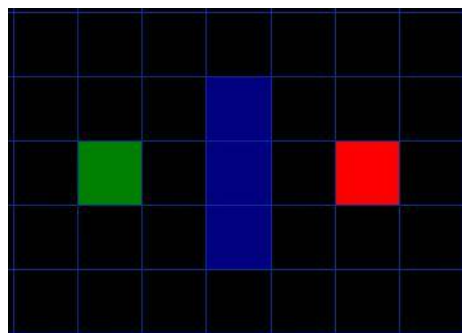
*Algoritmo de búsqueda A**

7	6	5	6	7	8	9	10	11		19	20	21	22
6	5	4	5	6	7	8	9	10		18	19	20	21
5	4	3	4	5	6	7	8	9		17	18	19	20
4	3	2	3	4	5	6	7	8		16	17	18	19
3	2	1	2	3	4	5	6	7		15	16	17	18
2	1	0	1	2	3	4	5	6		14	15	16	17
3	2	1	2	3	4	5	6	7		13	14	15	16
4	3	2	3	4	5	6	7	8		12	13	14	15
5	4	3	4	5	6	7	8	9	10	11	12	13	14
6	5	4	5	6	7	8	9	10	11	12	13	14	15

El **algoritmo de búsqueda A*** (A Asterisco) se clasifica dentro de los algoritmos de búsqueda en grafos. Presentado por primera vez en 1968 por Peter E. Hart, Nils J. Nilsson y Bertram Raphael, el algoritmo encuentra, siempre y cuando se cumplan unas determinadas condiciones, el camino de menor coste entre un nodo origen y uno objetivo.

Introducción: el área de búsqueda

Vamos a asumir que tenemos a alguien que quiere ir desde el punto A hasta el punto B. Asumamos también que un muro separa estos dos puntos. El ejemplo queda ilustrado en el gráfico siguiente, donde el recuadro verde es el punto de inicio A, el rojo es el punto de destino B y la zona azul el muro que hay entre ambos.



[Figura 1]

Lo primero que deberías advertir es que hemos dividido nuestra área de búsqueda en una rejilla cuadrada. Simplificar el área, tal y como hemos hecho, es el primer paso en un pathfinding. Este particular método reduce nuestra área de búsqueda a una simple matriz bidimensional. Cada elemento de la matriz representa uno de los cuadrados de la rejilla y su estado se almacena como transitable o intransitable. El camino se elige calculando qué cuadros deberíamos pisar para ir desde A hasta B. Una vez que el camino haya sido encontrado, el personaje de nuestro juego (o lo que sea) se moverá desde el centro de un cuadrado hacia el centro del siguiente hasta que el objetivo haya sido alcanzado.

A esos puntos centrales se les llama "nodos". Cuando leas en cualquier otro sitio sobre pathfinding, a menudo verás a gente hablando sobre nodos. ¿Acaso no es más fácil referirse a ellos como cuadrados? Esto se debe a que es posible dividir nuestra área en otras cosas aparte de cuadrados. Podrían ser rectangulares, hexagonales, o de cualquier otra forma; y podrían situarse en cualquier lugar dentro de esas formas - en el centro, por los bordes, o en cualquier lugar. Nosotros usamos este sistema porque es el más simple.

Iniciando la búsqueda

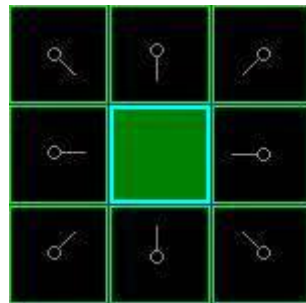
Una vez que hemos simplificado nuestra área de búsqueda en un número de nodos asequible, tal y como hemos hecho con la rejilla de la figura anterior, el siguiente paso es dirigir una búsqueda para encontrar el camino más corto. En el pathfinding A*, lo hacemos empezando desde el punto A, comprobando los cuadros adyacentes y generalmente buscando hacia fuera hasta que encontremos nuestro destino.

Empezamos la búsqueda haciendo lo siguiente:

1. Empezar en el punto inicial A y añadirlo a una "lista abierta" de cuadrados a tener en cuenta. La lista abierta es como una lista de la compra. Ahora mismo solo tenemos un elemento en la lista, pero más tarde tendremos más. La lista contiene los cuadrados que podrían formar parte del camino que queremos tomar, pero que quizás no lo hagan. Básicamente, esta es una lista de los cuadrados que necesitan ser comprobados.
2. Mirar en todos los cuadrados alcanzables o transitables adyacentes al punto de inicio, ignorando cuadrados con muros, agua u otros terrenos prohibidos. Añadirlos a la lista abierta también. Por cada uno de esos cuadrados, guardar el punto A como su "cuadrado padre". El cuadrado padre es muy importante para trazar nuestro camino. Lo explicaremos más tarde.

3. Sacar el cuadro inicial A desde tu lista abierta y añadirlo a una "lista cerrada" de cuadrados que no necesitan, por ahora, ser mirados de nuevo.

En este punto, deberías tener algo como la siguiente ilustración. En este diagrama, el cuadrado verde oscuro del centro es tu cuadrado de inicio. Está bordeado el azul claro para indicar que el cuadrado ha sido añadido a la lista cerrada. Todos los cuadros adyacentes están ahora en la lista abierta para ser comprobados, están bordeados con verde claro. Cada uno tiene un puntero gris que señala a su padre, el cual es el cuadro inicial.



[Figura 2]

Después, elegimos uno de los cuadrados adyacentes de la lista abierta y más o menos repetimos el proceso anterior como se describe un poco más abajo. Pero ¿cuál cuadro debemos elegir? Aquel que tenga el coste F más bajo.

Puntuando el camino

La clave para determinar qué cuadrados usaremos para resolver el camino está en la siguiente ecuación:

$$F = G + H$$

donde

- G = el coste de movimiento para ir desde el punto inicial A a un cierto cuadro de la rejilla, siguiendo el camino generado para llegar allí.
- h = el coste de movimiento estimado para ir desde ese cuadro de la rejilla hasta el destino final, el punto B. Esto es a menudo nombrado como la heurística, la cual puede ser un poco confusa. La razón por la cual es llamada así, se debe a que es una suposición. Realmente no sabemos la distancia actual hasta que encontramos el camino, ya que toda clase de cosas pueden estar en nuestro camino (muros,

agua, etc.) En este tutorial, estás viendo una forma de calcular H, pero hay muchas otras que podés encontrar en otros artículos de la red.

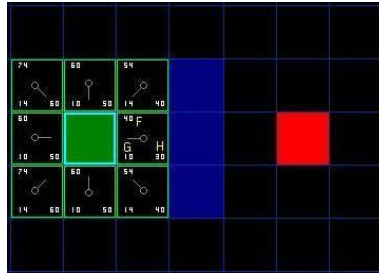
Nuestro camino se genera por ir repetidamente a través de nuestra lista abierta y eligiendo el cuadrado con la puntuación F más baja. Este proceso se describirá con más detalle un poco más adelante. Primero veamos más de cerca cómo calculamos la ecuación.

Tal y como está descrito más arriba, G es el coste de movimiento para ir desde el punto de inicio a un cierto cuadro usando el camino generado para llegar allí. En este ejemplo asignaremos un coste de 10 a cada cuadro vertical u horizontal hacia el que nos movamos, y un coste de 14 para un movimiento diagonal. Usamos estos números porque la distancia actual para mover diagonalmente es el cuadrado de la raíz de 2 (no temer), o más o menos 1,414 veces el coste del movimiento horizontal o vertical. Usamos 10 y 14 con el fin de simplificar. El rango es bastante bueno y así nos libramos de tener que calcular raíces cuadradas y sus decimales. Esto no es solo porque seamos idiotas y no nos gusten las matemáticas; usar números enteros también es mucho más rápido para el ordenador. Pronto descubrirás que el pathfinding puede ser muy lento si no usás atajos como este.

Ahora que hemos calculado el coste G mediante un camino específico hasta cierto cuadrado, la forma de resolver el coste G del cuadrado es tomar el coste G de su padre y luego añadirle 10 o 14 dependiendo de si está en diagonal u ortogonal (no diagonal) con respecto a ese cuadro padre. Este método se hará más claro cuando llevemos este ejemplo un poco más allá y nos alejemos un cuadro del inicial.

H puede ser estimado de diferentes maneras. El método que hemos usado aquí se llama el método Manhattan, donde calculás el número total de cuadros movidos horizontalmente y verticalmente para alcanzar el cuadrado destino desde el cuadro actual, sin hacer uso de movimientos diagonales. Luego multiplicamos el total por 10. Se llama método Manhattan porque es como calcular el número de manzanas que hay desde un lugar a otro, donde no podés acortar atravesando en diagonal una manzana. Cabe señalar que cuando calculamos H, ignoramos cualquier obstáculo que intervenga. Es una estimación de la distancia que queda, no de la distancia actual, es por eso que se llama heurística.

F se calcula sumando G y H. El resultado del primer paso en nuestra búsqueda puede verse en la ilustración inferior. Las puntuaciones F, G y H están escritas en cada cuadrado. En el cuadro inmediatamente a la derecha de cuadro inicial, la F está impresa arriba a la izquierda, la G abajo a la izquierda y la H abajo la derecha.



[Figura 3]

Así pues, vamos a mirar algunos ejemplos de estos cuadros. En el cuadrado con letras, $G=10$. Esto es debido a que está solo a un cuadro del cuadrado inicial en dirección horizontal. Los cuadrados inmediatamente encima, abajo y a la izquierda del cuadrado inicial; tienen todos el mismo valor G de 10. Los cuadros diagonales tienen un valor G de 14.

Las puntuaciones H se calculan estimando la distancia Manhattan hasta el cuadrado rojo objetivo, moviéndose solo horizontal y verticalmente e ignorando el muro que está en el camino. Usando este método, el cuadro de la derecha del inicial, está a 3 cuadros del cuadrado rojo con una puntuación H de 30. El cuadrado está a solo 4 cuadros de distancia (recordar que solo nos movemos en horizontal y vertical) con una puntuación H de 40. Probablemente comprendas cómo se calculan las puntuaciones H para los demás cuadros.

De nuevo, la puntuación F de cada cuadro se calcula simplemente sumando G y H .

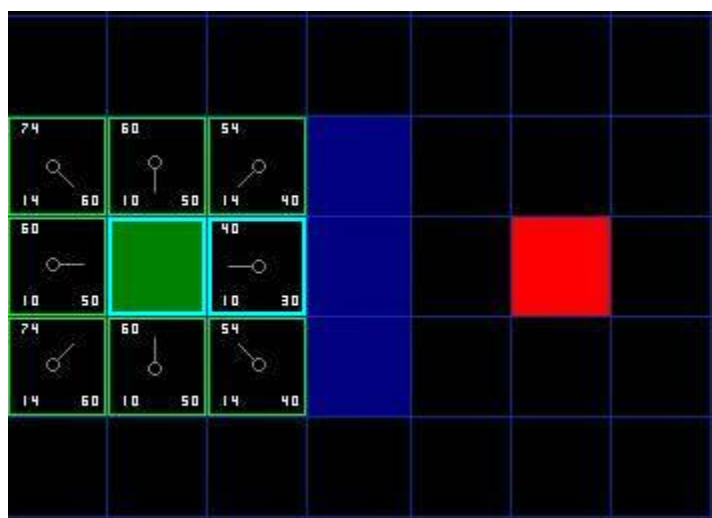
Continuando la búsqueda

Para continuar la búsqueda, simplemente elegimos la puntuación F más baja de todos aquellos que estén en la lista abierta. Después hacemos lo siguiente con el cuadro seleccionado:

4. Sacarlo de la lista abierta y añadirlo a la lista cerrada.
5. Comprobar todos los cuadrados adyacentes, ignorando aquellos que estén en la lista cerrada o que sean intransitables terrenos con muros, agua, o cualquier terreno prohibido), añadir los cuadros a la lista abierta si no están ya en esa lista. Hacer al cuadro seleccionado el "padre" de los nuevos cuadros.
6. Si el cuadro adyacente ya está en la lista abierta, comprobar si el camino a ese cuadro es mejor que este. En otras palabras, comprobar que la G de ese cuadro sea más baja que la del que estamos usando para ir allí. Si no es así, no hagas nada. Por otro lado, si el coste G del nuevo camino es más bajo, cambia el padre del cuadro adyacente al cuadro seleccionado (en el diagrama superior, cambia la

dirección del puntero para que señale al cuadro seleccionado). Finalmente, recalculamos la F y la G de ese cuadrado. Si esto te parece confuso, podrás verlo ilustrado más abajo.

Ahora vamos a ver cómo funciona. De nuestros 9 cuadros iniciales, dejamos 8 en la lista abierta después de que el cuadrado inicial fuera incluido en la lista cerrada. De estos, el que tiene el coste F más bajo es el de inmediatamente a la derecha del cuadro inicial, con una F de 40. Así que seleccionamos este cuadrado como nuestro siguiente cuadrado. Está resaltado en azul en la siguiente ilustración.



[Figura 4]

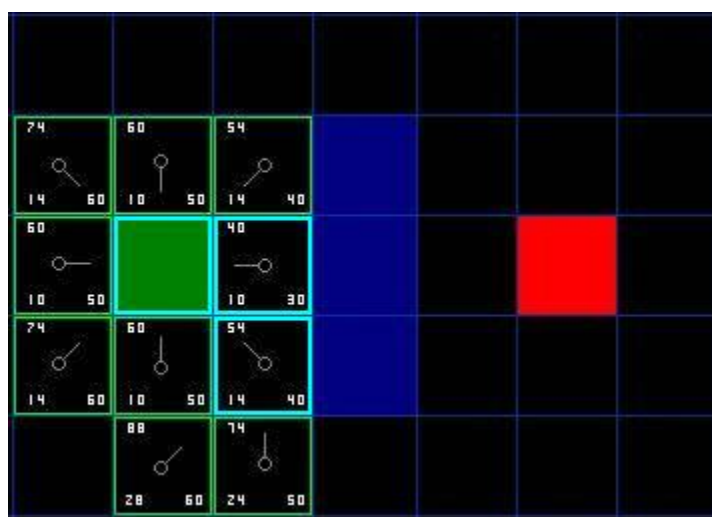
Primero, lo sacamos de nuestra lista abierta y lo añadimos a nuestra lista cerrada (por que ahora está resaltado en azul). Luego comprobamos los cuadros adyacentes. Todos los que hay a la derecha son cuadros de muro, así que no los tenemos en cuenta. El de la izquierda es el cuadrado inicial; ese está en la lista cerrada, así que lo ignoramos también.

Los otros 4 cuadros ya están en la lista abierta, así que necesitamos comprobar si alguno de los caminos hasta esos cuadros es mejor que el del cuadrado actual hasta ellos. Para eso usaremos la puntuación G como punto de referencia. Miremos debajo de nuestro cuadrado seleccionado; su G actual es 14. Si fuésemos a través del cuadrado actual hasta allí, la G sería igual a 20 (10, la G del cuadrado actual, más 10 de un movimiento vertical hacia el cuadro superior). Una G de 20 es mayor que una de 14, así que no es un buen camino. Todo eso debería cobrar sentido si mirás el diagrama. Es más directo llegar a ese cuadro desde el cuadro inicial moviéndote un cuadro en diagonal, que moverte horizontalmente un cuadro y luego verticalmente otro.

Cuando repetimos este proceso para los otros 4 cuadros adyacentes que ya están en la lista abierta, descubrimos que ninguno de los caminos ha mejorado por ir a través del cuadro actual (el de la derecha bordeada de azul), así que pasamos de él. Ahora que hemos mirado en todos los cuadros adyacentes y ya nos hemos hecho con este nuevo cuadro, estamos listos para movernos al siguiente cuadrado.

Recorremos el grupo de cuadros de nuestra lista abierta, ahora ha bajado a 7 cuadros, tomamos el que tenga el coste F más bajo. Interesante..., en este caso hay dos cuadros que tienen una puntuación de 54. Así que, ¿cuál elegimos? La verdad es que no importa. Para propósitos de velocidad, puede ser más rápido elegir el último que añadimos a la lista abierta. Esto influencia la búsqueda en favor de cuadros que fueron encontrados más tarde justo cuando estés más cerca de alcanzar tu objetivo. De todas formas, no es verdaderamente importante.

Así pues, y ya que lo elegimos anteriormente, escogemos el cuadro justo debajo del cuadrado que desechamos antes. La siguiente ilustración lo aclara:

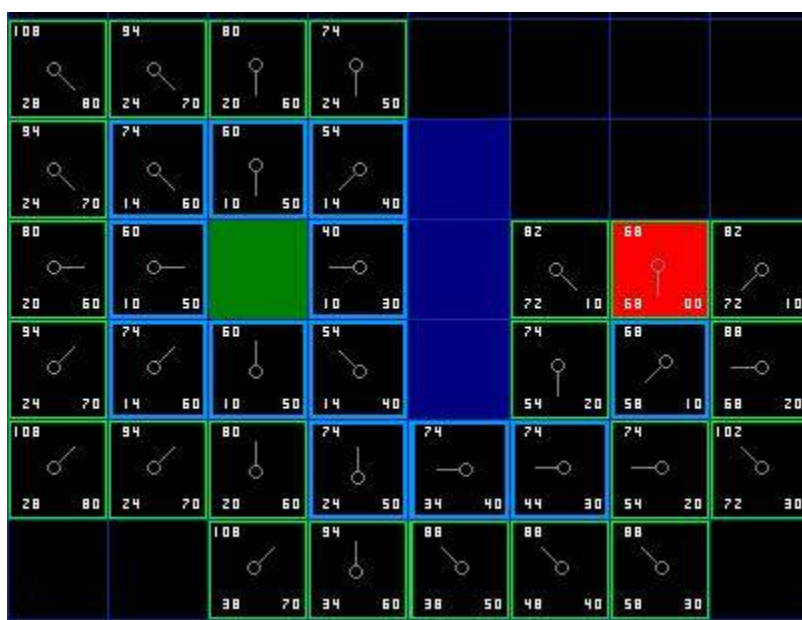


[Figura 5]

Esta vez, cuando comprobamos los cuadrados adyacentes encontramos que el de la izquierda y el superior a este son cuadros de muro, así que los ignoramos. Tampoco tenemos en cuenta el cuadro que está debajo del muro. ¿Por qué? Porque no podés llegar a ese cuadrado sin que tu personaje se raspe el hombro con la esquina al intentar pasar en diagonal; lo mejor es dar un pequeño rodeo, primero bajando y luego yendo hacia la derecha. (Nota: esta regla de bordear esquinas es opcional. Su uso depende de cómo estén situados tus nodos.)

Eso nos deja otros 5 cuadros. Los otros 2 cuadros bajo el actual no están en la lista abierta, así que los añadimos y el cuadro actual se convierte en su padre. De esos otros 3 cuadros, 2 ya están en la lista cerrada (el cuadro inicial, y el cuadro que hay encima del actual, ambos resaltados en azul en el diagrama) así que los ignoramos. El último cuadro, el de la izquierda del cuadro actual, se comprueba para ver si el coste G hasta él desde el cuadro actual, es menor que llegando directamente desde el cuadro inicial. Lo hacemos y no hay suerte, así que ya estamos listos para comprobar el siguiente cuadro de nuestra lista abierta.

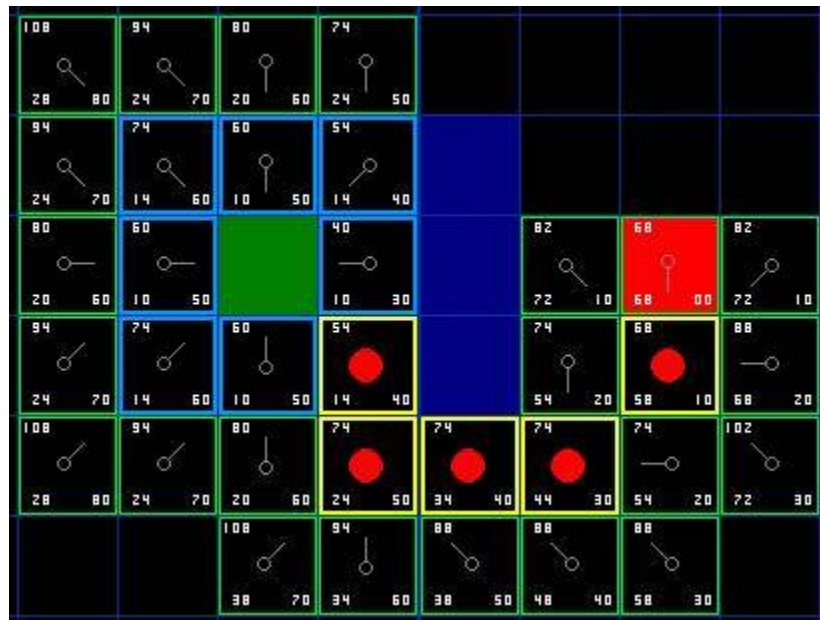
Repetimos este proceso hasta que añadimos el cuadro objetivo a la lista abierta, en ese momento parecería algo como la ilustración inferior:



[Figura 6]

Observar que el cuadro padre para el cuadrado dos cuadros por debajo del cuadro inicial ha cambiado desde la ilustración anterior. Antes tenía un coste G de 28 y apuntaba al cuadrado encima suya y a la derecha. Ahora tiene una puntuación de 20 y apunta al cuadrado encima suya. Esto ocurrió en algún momento por la forma en la que se ejecuta nuestra búsqueda, donde la puntuación G fue comprobada y devuelta más baja usando un nuevo camino - el padre cambió y G y F fueron recalculadas. A pesar de que este cambio no parece demasiado importante en este ejemplo, hay muchas posibles situaciones donde este constante control significará la diferencia en la determinación del mejor camino hasta tu objetivo.

Pero, ¿cómo determinamos el camino actual en sí mismo? Fácil, solo empezas desde el cuadro objetivo rojo y vas hacia atrás de un cuadrado a su padre, siguiendo las flechas. Eso te llevará de vuelta al cuadrado inicial y tus movimientos serán el camino a seguir. Debería parecerse a la siguiente ilustración. Moverse desde el cuadro inicial A hasta el cuadro destino B es solo cuestión de ir moviéndose desde el centro de un cuadro (el nodo) al siguiente hasta alcanzar el objetivo.



[Figura 7]

Sumario del Método A*

Ahora que has leído la explicación, vamos a resumir el método paso a paso:

1. Añadir el cuadro inicial a la lista abierta.
2. Repetir lo siguiente:
 - a) Buscar el cuadro con el coste F más bajo en la lista abierta. Nos referimos a este como el cuadro actual.
 - b) Cambiarlo a la lista cerrada
 - c) Para cada uno de los 8 cuadros adyacentes al cuadro actual:
 - Si no es transitable o si está en la lista cerrada, ignorarlo. En cualquier otro caso hacer lo siguiente.

- Si no está en la lista abierta, añadirlo a la lista abierta. Hacer que el cuadro actual sea el padre de este cuadro. Almacenar los costes F, G y H del cuadro.
 - Si ya está en la lista abierta, comprobar si el camino para ese es mejor usando el coste G como baremo. Un coste G menor significa que este es un mejor camino. Si es así, cambiar el padre del cuadrado al cuadro actual y recalcular G y F del cuadro. Si estás manteniendo la lista abierta por orden de puntuación F, podrías necesitar reordenar la lista para llevar cuenta del cambio.
- d) Para cuando:
- Añadas el cuadro objetivo a la lista abierta en cuyo caso el camino ha sido encontrado, o
 - Falles en encontrar el cuadro objetivo y la lista abierta esté vacía. En este caso no hay camino.
3. Guardar el camino. Moverse hacia atrás desde el cuadro objetivo, ir desde cada cuadro a su padre hasta que alcances el cuadro inicial. El camino seguido es el que buscás.

Observación

Es importante señalar que cuando leas varias discusiones sobre el pathfinding A* en la web y en diferentes foros, alguna vez verás que alguien se refiere a cierto código como A* cuando en realidad no lo es. Para el método A* necesitás incluir los elementos que hemos explicado arriba, específicamente listas cerradas y abiertas y puntuación del camino usando F, G y H. Hay muchos otros algoritmos de pathfinding, pero esos otros métodos no son A*, el cual es generalmente considerado como el mejor. Bryan Stout habla sobre muchos de ellos, incluyendo algunos de sus pros y contras. Algunas veces las alternativas son mejores bajo determinadas circunstancias, pero deberías comprender en lo que te estás metiendo.

Notas sobre la implementación

Ahora que comprendés el método básico, aquí tenemos algunos temas adicionales para cuando estés escribiendo tu propio programa. Algunos de los siguientes materiales hacen referencia al programa que he escrito en C++ y en Blitz Basic, de todas formas, los puntos son igualmente válidos en otros lenguajes.

1. Manteniendo la Lista Abierta: suele ser uno de los elementos que más tiempo consume en la función del pathfinding A*. Cada vez que accedes a la lista abierta necesitas encontrar el cuadrado que tiene el coste F más bajo. Hay varias formas en las que podrías hacerlo. Podrías guardar los elementos del camino según los necesites y simplemente recorrer la lista entera cada vez que necesites encontrar el cuadrado con el coste F más bajo. Esta es fácil, pero es una manera muy lenta para caminos largos. Podés mejorarlo manteniendo una lista ordenada y simplemente tomando el primer elemento de la lista cada vez que necesites el cuadro con el coste F más bajo.

Aunque esto funciona razonablemente bien para pequeños mapas, no es la solución más rápida. Programadores serios de A* que buscan más velocidad, usan algo llamado Heap binario que es lo que usé en mi código. En mi opinión, es aproximadamente de 2 a 3 veces más rápido que el orden estándar en la mayoría de las situaciones, y también geométricamente más rápido (+ de 10 veces) en caminos largos.

2. Otras Unidades: si se te ocurre examinar mi código de ejemplo, verás que ignora completamente otras entidades que haya en el mapa. Los criterios en los que se basa mi pathfinding consisten en ir derechos desde un punto A hasta uno B. Dependiendo del juego, quizás vaya bien o no. Si querés considerar otras entidades en el mapa y que las entidades interactúen entre sí, te sugiero que no tengas en cuenta esas nuevas entidades en el código del pathfinding y que lo añadas como código nuevo y externo. (Consideramos entidades a todo aquello que pueda influir en el camino de un personaje). Imaginar, por ejemplo, que el personaje debe ir hasta un punto del mapa y en su camino se tropieza con otro personaje, tal vez en casos como este quieras que tu personaje genere un nuevo camino o que use algunas reglas estándar de movimiento (Por ejemplo, que siempre lo rodee moviéndose hacia la derecha, o que si este personaje es dañino, se aleje) hasta que el obstáculo ya no se encuentre en su camino para generar de nuevo otro. ¿Por qué no incluir esas entidades cuando estamos calculando el camino inicial? Porque esas entidades quizás estén dotadas de movimiento y cuando llegues a su posición puede que ya no estén. Por este motivo, tenerlas en cuenta podría generar extraños resultados; tal vez esquivando en su camino a una entidad que ya no está allí o "pisoteando" a aquellas que se desplazaron a una situación dentro del camino tras haber sido calculado.

No obstante, ignorar otras unidades dentro del pathfinding significa que necesitarás escribir código por separado para manejar colisiones. Este apartado es específico de cada juego, así que dejo la resolución del problema a tu criterio.

3. Algunos trucos para obtener más velocidad: cuando desarrolles tu propio programa A* o adaptes el que he escrito, quizás te des cuenta de que el pathfinding está consumiendo gran parte del tiempo de proceso de tu CPU. Esto es particularmente notorio si tienes un mapa (razonablemente) grande y un número decente de personajes en él. Si lees el material existente en Internet, verás que esto les pasa incluso a los profesionales que diseñan juegos como Starcraft o Age of Empires. Si ves que algunas cosas empiezan a ir más despacio por el pathfinding, aquí tienes algunas ideas que podrían acelerarlas:

- Considera la utilización de un mapa pequeño o unos pocos personajes.
- Nunca busques el camino para más de unos pocos personajes al mismo tiempo. En vez de ello, ponlos en una cola y realiza las búsquedas por grupos en varios ciclos de juego. Si tu juego está funcionando en, por ejemplo, 40 ciclos por segundo, nadie lo notará. Pero cuando un buen puñado de personajes estén intentando buscar simultáneamente su camino, te aseguro que tu programa irá más lento que una tortuga.
- Considera el uso de cuadrados más grandes para tu mapa. Esto reduce el número total de cuadrados que serán comprobados para encontrar el camino. Si sos ambicioso, puedes idear 2 o más sistemas de pathfinding que se usen en diferentes situaciones dependiendo de la longitud del camino. Esto es lo que hacen los profesionales, usan áreas grandes para pathfinding largos, y luego afinan la búsqueda con áreas más pequeñas cuando se acercan al objetivo.
- Considera el uso de un sistema de waypoints (paradas) para caminos largos o ingenia caminos precalculados que estén incluidos en el juego.
- Considera "preprocesar" tu mapa para calcular qué áreas son inaccesibles desde el resto del mapa. Yo llamo a esas áreas "islas". En realidad, pueden ser islas o cualquier área que esté amurallada y sea inaccesible. Uno de los inconvenientes de A* es que si le decís que busque caminos en un área, él los buscará teniendo en cuenta el mapa entero, deteniéndose solo cuando cada nodo/cuadro accesible haya sido procesado mediante las listas abierta y cerrada. Eso puede desperdiciar mucho tiempo de CPU. Es posible prevenirlo determinando que áreas son inaccesibles (vía flood-fill o rutina similar), almacenando esa información en una matriz de algún tipo y luego comprobándolo antes de empezar a buscar el camino. En la versión Blitz de mi código, he creado un mapa "preprocesado" que hace esto. También están preidentificados los callejones sin salida que el algoritmo pathfinding puede ignorar, cosa que acelera bastante las cosas.

4.- Variable de Coste del Terreno: en este tutorial y en mi programa acompañante, el terreno es solo una de dos cosas - transitable o intransitable. ¿Pero qué pasa si tenés un terreno transitable, pero con un coste de movimiento mayor? Pantanos, colinas, escaleras en un dungeon, etc... - son todos ejemplos de terrenos transitables, pero con un coste de movimiento mayor que el de un terreno liso y abierto. Igualmente, una carretera podría tener un coste de movimiento menor que el terreno circundante.

Este problema se soluciona fácilmente añadiendo el coste del terreno cuando calcules el coste G del cuadro señalado. Simplemente añadí un coste extra a cada cuadro. El algoritmo A^* ya está escrito para encontrar el camino con el coste más bajo y manejarlo fácilmente. En el simple ejemplo que he descrito, cuando el terreno es solo transitable o intransitable, el A^* busca el camino más corto y directo. Pero en un entorno con un terreno de coste variable, el camino con un coste menor podría significar viajar una mayor distancia - como tomar una carretera que rodea un pantano en vez de atravesarlo directamente.

Una interesante consideración adicional es lo que algunos profesionales llaman "mapa de influencia". Al igual que el terreno de coste variable, podrías crear un sistema de puntuación adicional y aplicarlo a los caminos para propósitos de IA. Imaginá que tenés un mapa con un buen puñado de malhechores defendiendo un paso a través de una región de montaña. Cada vez que el ordenador enviase a alguien por un camino que atravesase el paso, lo enviaría a una muerte segura. Si querés, podrías crear un mapa de influencia que penalice los cuadros donde se esté produciendo alguna carnicería. Esto podría enseñar al ordenador caminos seguros y le ayudaría a evitar situaciones estúpidas en las que envía tropas y personajes según un camino en particular y solo porque es el más corto (pero también el más peligroso).

5.- Manejando áreas Inexploradas: ¿Has jugado alguna vez a un juego de PC donde la computadora siempre sabe exactamente qué camino tomar, aunque el mapa no haya sido explorado todavía? Dependiendo del juego, un pathfinding que sea demasiado bueno puede ser irreal. Afortunadamente, es un problema que puede resolverse con absoluta facilidad.

La solución es crear una matriz "traspasabilidadConocida" aparte para cada uno de los jugadores y oponentes controlados por la computadora (cada jugador, no cada unidad - ya que requeriría mucha más memoria). Cada matriz contendría información sobre las áreas que el jugador ha explorado, con el resto del mapa considerado como transitable hasta que se demuestre lo contrario. Usando esta aproximación, las unidades no evitarán

callejones sin salida y harán elecciones erróneas similares hasta que aprendan el camino. Sin embargo, una vez que el mapa esté explorado, el pathfinding funcionará normalmente.

6. Caminos más suavizados: aunque un A* te dará automáticamente el camino más corto y de coste menor, no te dará automáticamente el camino más suavizado. Echale un vistazo al camino final calculado en nuestro ejemplo (figura 7). En ese camino, el primer paso es en diagonal hacia abajo y a la derecha del cuadro inicial. ¿No sería nuestro camino más suavizado si el primer paso en vez de ser en diagonal fuese directamente hacia abajo?

Hay varias formas de tratar este problema. Cuando estés calculando el camino podrías penalizar cuadros donde hay un cambio de dirección, añadiendo una penalización de sus puntuaciones G. Alternativamente, podrías recorrer tu camino tras haberlo calculado, buscando lugares donde elegir un cuadro adyacente le daría un aspecto mucho mejor.

7.- Áreas de Búsqueda No Basadas en Cuadros: en nuestro ejemplo, hemos usado una simple capa de cuadros 2D. No estás obligado a hacer uso de lo mismo, podrías usar áreas de formas irregulares. Pensá en el tablero del Risk y en los países en el juego. Podrías inventar un pathfinding para el escenario de un juego como ese. Para conseguirlo, necesitarías crear una tabla para almacenar que países son adyacentes entre sí y un coste asociado con el movimiento de un país al siguiente. También necesitarías dar con un método para calcular H. El resto de las cosas serían iguales que en el ejemplo de arriba. En vez de usar cuadros adyacentes, simplemente buscarías países cercanos cuando añadas nuevos elementos a tu lista abierta.

Igualmente, podrías crear un sistema de waypoints (paradas) para los caminos en un mapa de terrenos adjunto. Los waypoints son normalmente puntos en una ruta (como las paradas de un autobús o metro), quizás en una carretera o en un túnel clave de un dungeon. Como diseñador de juego, podrías preasignar esos waypoints. Dos waypoints serían considerados "adyacentes" entre ellos si no hay obstáculos en la línea recta que les separa. Al igual que en el ejemplo del Risk, deberías almacenar esta información de adyacencia en una tabla de alguna clase y usarla cuando generes una nueva lista de elementos. Después deberías guardar los costes G asociados (quizá usando la línea directa entre nodos) y los costes H (tal vez usando una línea directa desde el nodo hasta el objetivo). El resto funcionaría igual que en el ejemplo de la explicación principal.

Minimax

En teoría de juegos, **Minimax** es un método de decisión para minimizar la pérdida máxima esperada en juegos con adversario y con información perfecta. Minimax es un algoritmo recursivo.

El funcionamiento de Minimax puede resumirse como elegir el mejor movimiento para vos mismo suponiendo que tu contrincante escogerá el peor para vos.

Teoría Minimax

John von Neumann es el creador del teorema Minimax, quien dio la siguiente noción de lo que era un juego:

"Un juego es una situación conflictiva en la que uno debe tomar una decisión sabiendo que los demás también toman decisiones, y que el resultado del conflicto se determina, de algún modo, a partir de todas las decisiones realizadas".

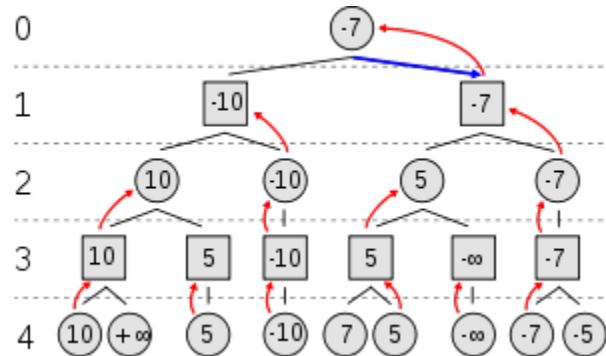
También afirmó que:

"Siempre existe una forma racional de actuar en juegos de dos participantes, si los intereses que los gobiernan son completamente opuesto".

La demostración a esa afirmación se llama Teoría Minimax y surge en 1926.

Este teorema establece que, en los juegos bipersonales de suma nula, donde cada jugador conoce de antemano la estrategia de su oponente y sus consecuencias, existe una estrategia que permite a ambos jugadores minimizar la pérdida máxima esperada. En particular, cuando se examina cada posible estrategia, un jugador debe considerar todas las respuestas posibles del jugador adversario y la pérdida máxima que puede acarrear. El jugador juega, entonces, con la estrategia que resulta en la minimización de su máxima pérdida. Tal estrategia es llamada óptima para ambos jugadores solo en caso de que sus minimaxes sean iguales (en valor absoluto) y contrarios (en signo). Si el valor común es cero, el juego se convierte en un sinsentido.

Algoritmo Minimax con movimientos alternativos



Pasos del algoritmo Minimax:

1. Generación del árbol de juego. Se generarán todos los nodos hasta llegar a un estado terminal.
2. Cálculo de los valores de la función de utilidad para cada nodo terminal.
3. Calcular el valor de los nodos superiores a partir del valor de los inferiores. Alternativamente se elegirán los valores mínimos y máximos representando los movimientos del jugador y del oponente, de ahí el nombre de Minimax.
4. Elegir la jugada valorando los valores que han llegado al nivel superior.

El algoritmo explorará los nodos del árbol asignándoles un valor numérico mediante una función de evaluación, empezando por los nodos terminales y subiendo hacia la raíz. La función de utilidad definirá lo buena que es la posición para un jugador cuando la alcanza. En el caso del ajedrez los posibles valores son (+1,0,-1) que se corresponden con ganar, empatar y perder respectivamente. En el caso del backgammon, los posibles valores tendrán un rango de [+192,-192], correspondiéndose con el valor de las fichas. Para cada juego pueden ser diferentes.

Si Minimax se enfrenta con el dilema del prisionero escogerá siempre la opción con la cual maximiza su resultado suponiendo que el contrincante intenta minimizarlo.

Bibliografía de referencia

- Russell, S. J. (2009). *Inteligencia artificial: un enfoque moderno* (3ª ed.) México: Prentice Hall Hispanoamericana.